# 第六章 系统体系结构与程序设计

2018年10月12日 9:27

- 系统设计包括两方面的主要问题——系统的体系结构设计和系统中程序的详细设计
  - 体系结构设计：把系统分成模块，（毕业论文一开始顶多是用例组，功能组，出现模块只能在设计中），在其中框架设计最重要
  - 程序详细设计：为每个模块开发完整的算法和数据结构，描述用例中的协作模型

1. 分布式物理体系结构Distributed physical architecture
   - 主要目标是降低耦合
   - 体系结构设计（具有物理和逻辑两个方面）
     i. 物理体系结构
        1) ——解决C/S 客户机/服务器问题，以及CS之间所需要的任何中间件的问题
        2) 使用结点和部署图
     ii. 逻辑体系结构
        1) 客户机client：发起请求
        2) 服务器server：处理客户端发来的请求
   - 使用CS体系结构进行扩展就能表示任意的分布式系统，在分布式系统中，客户机能够访问任意多台服务器，然而在同一时间，只允许客户机访问一台服务器，即在一个请求中不可能将来自两个或更多数据库服务器的数据组合起来，除非这是分布式数据库系统

a. 对等体系结构P2P
   i. 好处：
      1) 没有管理结点，使网络负载得到均衡
      2) 对于网络故障和单个同位体故障，对等系统易于快速恢复，且易于扩展和适应
   ii. 坏处：在对等网体系结构中，当总的系统吞吐量达到最大时，就要考虑网络流量最小化的问题。确保对等系统具有一致的反应时间是一种挑战。
   iii. 主要用在分布式的文件共享系统（六维空间）
   iv. 系统中任何结点可能既是客户机，也是服务器

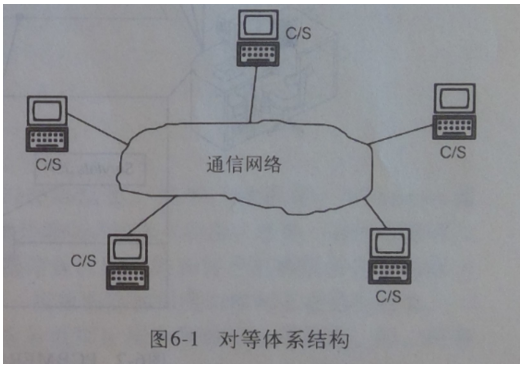


图6-1 对等体系结构

b. 分层体系结构——为了降低耦合
   i. 像同位体的情况一样，层次结构中间的每一层既是客户机也是服务器
   ii. 在硬件层于软件层之间采用层次依赖方法
   iii. 如三层体系结构——GUI客户机、单独的业务逻辑、数据库服务器

iv. **应用服务**是每一个运行在数据库上的单独程序所做的，**业务服务**是数据库强迫所有运行程序都要遵循的业务规则，这种业务规则的强制执行通常是通过编程实现的（运用数据库触发器和存储过程）

v. 当应用逻辑被编译到客户机时——胖客户体系结构“a client on steroids”

vi. 当应用逻辑被编译到服务器时——瘦客户体系结构“a skinny client”

vii. 一部分应用逻辑被编译到客户端，一部分被编译到服务器——中介者体系结构

c. 以数据库为中心的体系结构

i. 数据库程序被称为存储过程，可以从客户端程序调用它

ii. 一种特殊的存储过程（触发器）不能被显示地调用。当试图更改数据库内容时，触发器自动触发

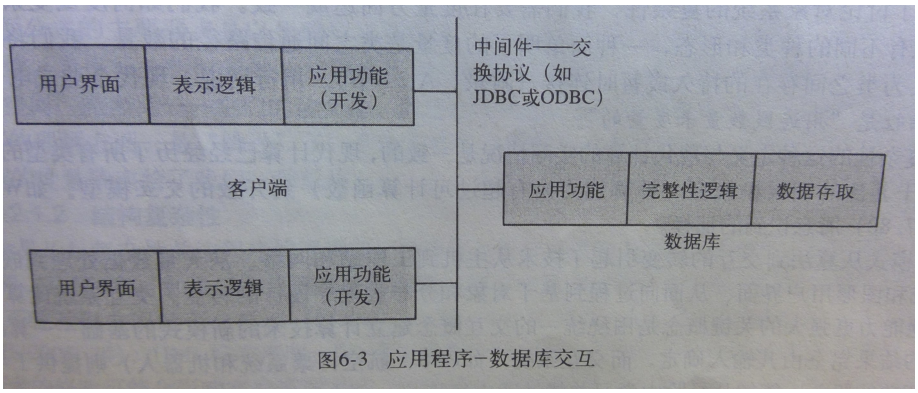


图6-3 应用程序-数据库交互

2. 多层逻辑体系结构Mulitilayer logical

○ 成功的系统通过层次结构hierarchies组织，将指数级复杂度降低成多项式级

a. 体系结构的复杂性

○ 两种主要的、有差别的计算模型是基于算法的图灵机模型和基于算法和开放的交互模型

○ 复杂度概念

□ 问题复杂性：由问题域本身复杂性定义（软件本质特性的一个分支）

□ 算法复杂性：目标是度量算法的效率

□ 结构复杂性：建立软件结构之间的关系及易于维护和演化，即降低耦合度的问题

□ 认知复杂性（Cognitive complexity）：理解软件做出的努力，即捕获程序的逻辑流，理解软件所需要的工作量的测量

○ 认知复杂性度量可以增强可理解性度量，结构复杂性度量可以增强可维护性和可伸缩性度量

i. 空间认知复杂性——度量软件工程师能看懂代码目光所移动的代码的行数 (measure the distance that a software engineer must move within the code to build a mental model of it. )

导出复杂性的值需要两步。第一步，计算程序中每个函数的复杂性值；第二步，将这些值累加得到整个程序的复杂性值。

调用次数

$$FC = \sum_{i=1}^{totalcalls} dist_i \quad (6\text{-}1)$$

其中：*totalcalls*是调用函数的次数；*dist*是距离，指从函数调用到函数定义的代码行数；*FC*是函数空间复杂性。

函数个数

$$PC = \sum_{i=1}^{totalfunctions} FC_i \quad (6\text{-}2)$$

其中：*totalfunctions*是程序中函数的数量；*PC*是程序的空间复杂性。

ii. 结构复杂性——强调程序对象之间的依赖关系，尽量减少依赖

□ 如果系统中所有的依赖都被标识和理解，则系统具有可适应性（可理解、可维护、可扩展）

1) 网络结构复杂性
   a) 每条通信路径允许类之间的双向交互——从B到A和从A到B
   b) 计算公式：

$$_{net}CCD = n(n-1)$$

其中：$n$是对象（图中的结点）的数量；
$_{net}CCD$是在完全连接的网络中累计的类依赖（假设对象指的是类）。

   c) 举例：

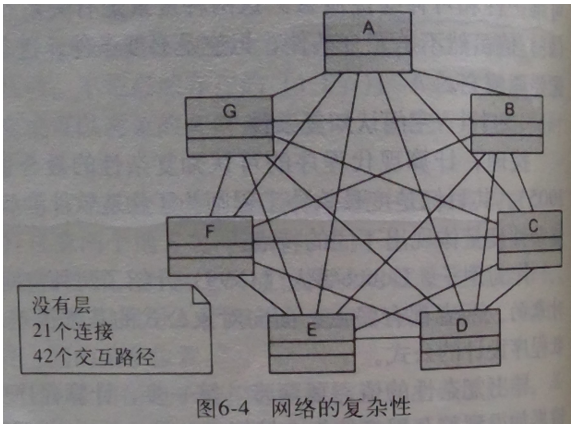


图6-4 网络的复杂性

2) 层次结构复杂性
   a) 强调层之间的层次体系分解，而在层的内部则允许类似网络的交互
   b) 通过一系列分层原则，如PCBMER中的DDP（向下依赖）原则，即层之间的通信路径是向下的
   c) 计算方法：设层l1,l2,l3,...,ln.对于任一层li，令
      i) size(li)为第li层的对象数
      ii) li为第li层的双亲数
      iii) pj(li)为第li层的第j个双亲

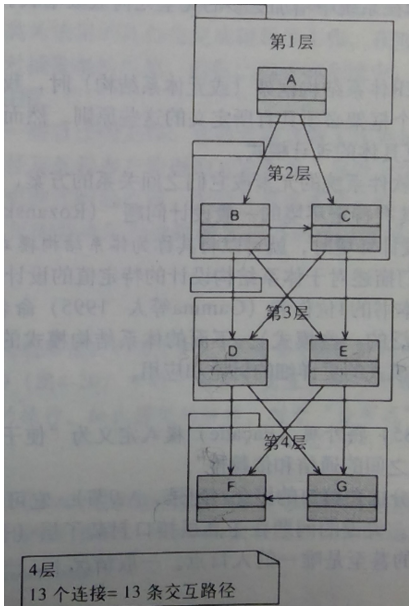
然后，根据式(6-4)计算一个合并结构的累计类依赖CCD（即允许多个双亲层的层次体系，如在PCBMER框架中一样）：

$$_{holarchy}CCD = \sum_{i=1}^{n} \frac{size(l_i)\times(size(l_i)-1)}{2} + \sum_{i=1}^{n}\sum_{j=1}^{l_i}(size(l_i)\times size(p_j(l_i))) \quad (6\text{-}4)$$

其中，$n$是每一层$i$（即第1层、第2层、第3层等）中类的数量，$_{hier}CCD$是层次体系中的累计类依赖。

式(6-4)中的第1部分计算每一层内所有类之间的潜在（所有可能的）单向路径的数量，第2部分计算每一对相邻层的类之间的潜在（所有可能的）单向路径的数量。

   d) 举例：

i) 


ii) 为啥是13不是12?

iii) 因为此图显示的是特定情形，CCD计算的是普遍最坏可能值

One. 第一层连接数为0

Two. 在2，3，4层内加1个连接（共3个）

Three. 在1，2层间加2个连接

Four. 在2，3层间加4个连接

Five. 在3，4层间加4个连接

b. 体系结构模式（设计模式/模板被用于体系结构设计，提供通用方法）

- 是什么，怎么用，用了这个模式有什么区别

i. 外观（facade）

1) 目标使用外观模式来对层使用简单的接口，降低子系统之间的耦合或依赖
2) 副作用：层中的类隐藏在外观对象的后面，需要更复杂选择的客户仍然可以直接与层中的类通信
3) 使子系统更易于使用的高层接口，这个高层接口可以是具体类（域类）或抽象类
4) 关键的是高层接口封装了高层（子系统，包）的主要功能，并向该层的客户提供主要的甚至是唯一的入口点。一般情况下，一个层会针对更高层的不同客户定义多个外观。

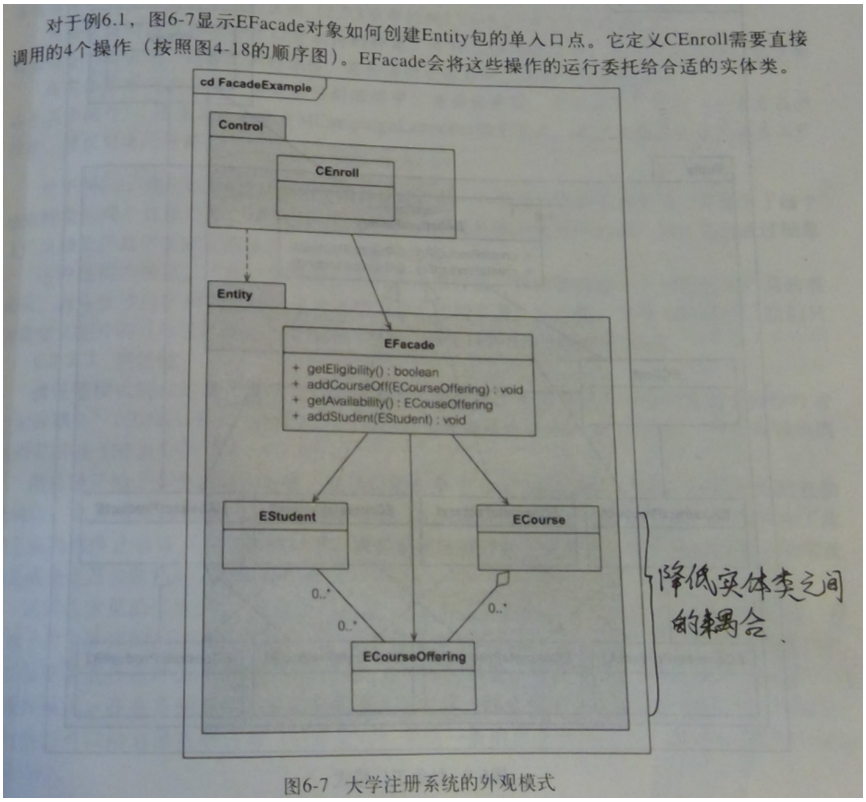


图6-7 大学注册系统的外观模式

ii. 抽象工厂（abstract factory）
   - 有益于增强应用系统的整体灵活性。这种灵活性在设计期间和运行期间都有体现
   1) 一个工厂通过参数的调整可以生产不同类型的产品（族）
   2) 可理解为外观模式的变种，在外观模式中，高层接口将意味着一个具体类，而抽象工厂中的接口或者是一个真正的接口（抽象类）
   3) 比如抽象工厂泛化为高端工厂和低端工厂，高端工厂生产高端产品族（高端轮胎、方向盘），低端同理

iii. 责任链（chain of Responsibility），与递归类似
   - 在PCBMER中，责任链使得NCP（相邻通信原则）称为可能
   1) 避免发送者和接收者之间的多重耦合
   2) 用双亲-子或容器-被包含模型实现，如果子对象没有能力处理该消息，它将返回给双亲或者是双亲的双亲，直到找到一个合适的处理对象

iv. 观察者（observer/listener），也叫出版-订阅模式
   - 两类对象：
     - 被观察对象：称为主题或者出版者
     - 观察对象：称为观察者、订阅者或监听者
   - 一个主题可以由很多观察者订阅他，主题状态的变更会通知到所有的观察者，然后所有的观察者都会执行必要的处理，使他们的状态与主题的状态同步。观察者之间没有相互关联，并且可以做不同的处理来响应主题状态变更的通知
   1) 定义对象间的一对多依赖，使得当一个对象变更状态时，其所有依赖者都被通知到并自动更新
   2) 比如全家人盯着一条刚买的不听话的狗
   - 对应的相反的模式，通知模式，由主动观察者去被动接收被观察者发来的消息，如狗饿了向全家人叫唤

v. 中介者（mediator）

1) 定义类来封装其他类之间的相互通信。通过使对象之间的不显示引用来降低耦合，并允许独立地改变他们的交互
2) 它允许我们将复杂的处理规则组织到专门的中介者类中，从而使其他复杂的对象不需要交换很多消息就可以做这种处理

3. 体系结构建模
   a. 包
      - 用于表示一组类，使高度相关的类的聚合
      - 包可以嵌套（nested）
      - 包之间的关联：泛化和依赖
      - 一个类只能在一个包中定义，但是可以通过引用出现在其他包中
      - 包之间的泛化也意味着依赖，依赖从子类包指向超类包
   b. 构件（了解，知道）
      - 是系统的物理部分，是一个独立的部署单元，即构件里面的部分不能再分了
      - 是第三方组装单元——它是充分文档化和自包含，可以被第三方插入到其他构件中
      - 构件没有持久状态——不能与他的拷贝区分开来，在任何给定的应用系统中，最多只有一个特定的构件有拷贝
      - 可替换性——相同接口的构件可以互相替换
      - 构件可以完成清晰的功能，并且是逻辑耦合和物理耦合
      - 可以嵌套在其他构件中

      i. 构件与包
         1) 从逻辑上讲，每一个类属于一个单一的包
         2) 从物理上讲，每一个类都至少由一个构件实现，并且一个构件可能只实现一个类。
            a) 抽象类和接口经常被多个构件实现
         3) 包是比构件更大的体系结构单元，倾向于以水平horizontal方式组织类——在应用域中静态接近的类
         4) 构件是对行为相近的类的垂直组织——这些类可能来自不同的域，但贡献于一个单一的业务活动（如用例）

      ii. 构件与类的接口
         1) 构件最终也来实现接口，构件只显示它所包含的类的某些接口，很多其他接口都被封装在构建中，只被协作的类在内部使用，对于其他构件是不可见的
         2) 构件是部署在某个计算机结点上的物理抽象
         3) 类表示逻辑事务，为了起到物理抽象的作用，不得不将其实现为构件

      iii. 结点——其实是一系列机器，服务器，计算机，客户端等
         1) 在UML中，分布式物理体系结构或系统的任何其他体系结构都被描述为部署图，部署图中的计算资源（运行时的物理对象）称为结点
         2) 组件是最终要部署到某个结点上的，即结点是构件的一个载体（结点是硬件，构件是软件）

4. 程序设计与复用原则

- 系统设计
  - 架构设计——框架
  - 详细设计——前界面，后数据库
  - 程序设计——对界面设计和数据库设计的扩展

a. 类的内聚cohesion与耦合coupling
   - 良好的程序设计可以确保类的内聚和耦合之间的良好平衡
   - 类内聚是一个类内部自确定的程度，它测量类独立的强度，内聚越强越好
   - 类耦合是类之间连接的程度，它测量类的相互依赖性，耦合越弱越好，但为了“协作”不得不耦合
   - 内聚和耦合是彼此关联，更强的内聚导致更弱的耦合，需要在两者之间取得最好的平衡，一些启发原则
     - 两个类或者不彼此依赖，或一个类只依赖另一个类的公共接口
     - 属性和相关方法应该保存在一个类中，如学校注册只关注学生的学号姓名等属性
     - 一个类应该只捕获一个对象,无关信息移到另一个类中
     - 系统功能应该尽可能均匀分布

   i. 类耦合的种类
      - 两个类为了通信，就需要耦合。如果类A直接引用类B，则类A和类B之间的耦合就存在
      - 耦合的6种常见形式：
        - X包含Y，或X具有指向Y的实例的属性，即X里的某个属性的类型就是Y
        - X具有引用Y的实例的方法，如使用类型Y的参数或局部变量，或一个消息返回类型Y的对象给X
        - X调用Y的服务（给Y发送消息）
        - X是Y的直接子类或非直接子类
        - X具有输入参数是类Y的方法
        - Y是一个接口，X实现了此接口

   ii. Detemer法则，耦合规则
      1) 类耦合应该尽量被限制在类的层次内，即层内耦合。层间耦合应该被最小化并小心引导
      2) 该法则说明了在类方法中允许什么样的消息目标：
         a) 方法的对象本身——C#和Java中的this，Smalltalk中的self和super
         b) 方法型构中作为参数的一个对象
         c) 此对象的属性所引用的对象（包括属性集合中所引用的对象）
         d) 此方法创建的对象
         e) 全局变量引用的对象
      3) 为了限制继承来的耦合，使用Demeter增强法则，将c)规则限制在类本身定义的属性上

   iii. 存取方法Accessor methods和机械类mindless classes
      - 存取方法定义观察者get和改变者set的操作
      - 一个具有很多存取方法的类称为机械类，也称为无思维类，由其它类决定该类状态

iv. 动态分类和混合实例内聚

1) Dynamic classification (If object can dynamically change its class) not supported ->mixed-instance cohesion
2) 没有动态分类的支持造成了混合实例内聚，这是面向对象编程环境的弱点
3) 带有混合实例内聚的类中的某些对象可能是未定义的，这时候可用泛化消除这个问题

b. 复用策略

- 策略意味着复用的粒度（Resue granularity）
  - 类
  - 构件component
  - 解决方案
- 与粒度相关联，对应3种复用策略

i. 工具包复用Toolkits(class libraries)

1) 强调类级的代码复用
2) 两种类型的工具包
   a) 基础工具包（内置好的），基础类由对象编程环境提供包括String、Date等
   b) 体系结构工具包，作为系统软件的一部分，如操作系统、数据库软件
      i) 在应用范围内与应用相关的包，靠个人积累

ii. 框架复用Frameworks

1) 框架强调构件级的复用
2) 一个框架是可定制的应用软件

iii. 模式复用Analysis and design patterns

1) 模式强调在开发方法过程中的复用

5. 协作建模Collaboration modeling

- 复合结构指相互关联的元素组合

a. 协作

i. 协作描述相互协作的角色（元素）结构，每个元素执行特定的功能，这些元素共同完成某项期望的功能

ii. 协作被形象地表示为一个带有协作名字的虚椭圆，矩形图标表示协作的实体（角色），角色由连接器connector连接，在协作名字的后面可以写上分类器名。

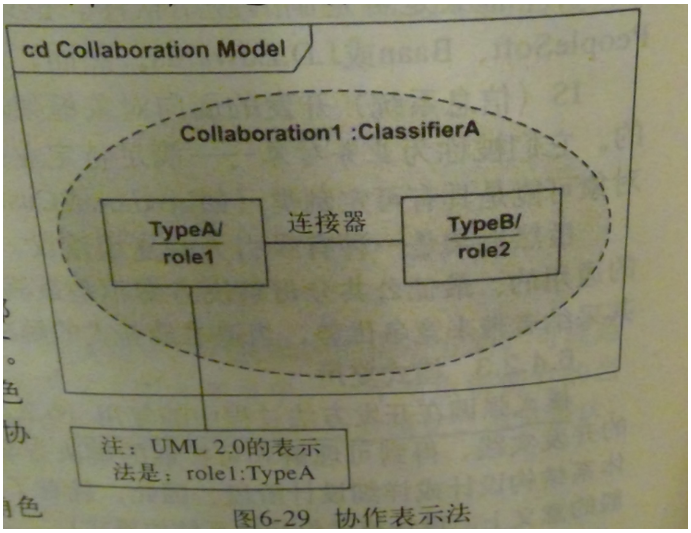


图6-29 协作表示法

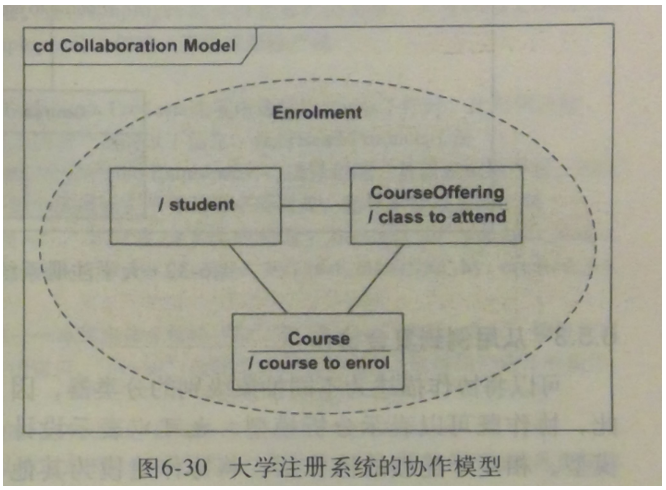


图6-30 大学注册系统的协作模型

iii. 一个角色可以有一个类型，角色的实例被绑定到此类型。类型是一个分类器，通常是一个类，角色中类型的显示是可选的。只有在协作中角色才是有意义的，同一个对象/实例可以在不同的协作中扮演不同的角色。连接器也只在协作中有意义

b. 复合结构——另一种类图的创建方法

i. 将协作转换成复合结构，需要将所有的角色从椭圆中抽取出来放在类种（因此需要为角色定义任何缺失的类型），将协作连接到类上，并为连接器定义上角色名（role1、role2）

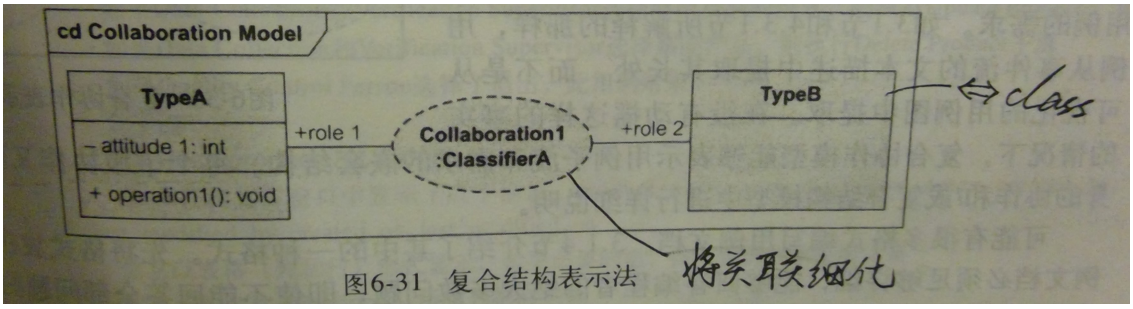


图6-31 复合结构表示法

c. 从用例到复合协作

i. 复合协作由下级协作组成（实现），一个复合协作可以实一个用例，而下级协作可以表示此用例的需求

为上面所给出的详细用例描述构造复合协作是一项简单的练习，将用例当成复合协作并抽取子流到下级协作中，如图6-34所示。

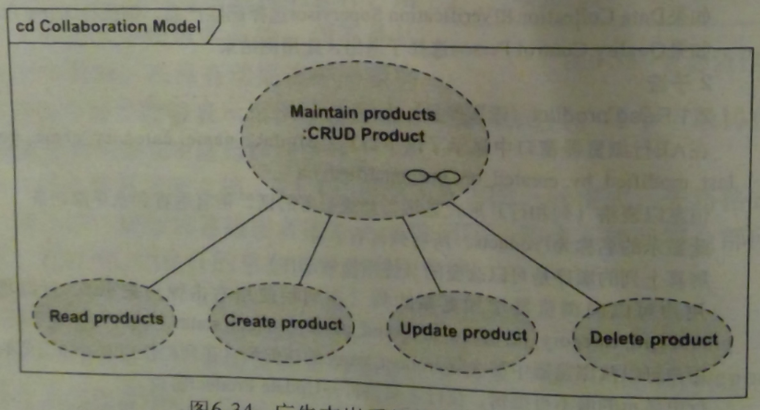



图6-34 广告支出系统的复合协作模型

d. 从协作到交互

i. 协作可以被用于生成顺序图和通信图的手段。为此，协作角色称为了顺序图上的生命线，连接器被交互中的消息代替。

ii.

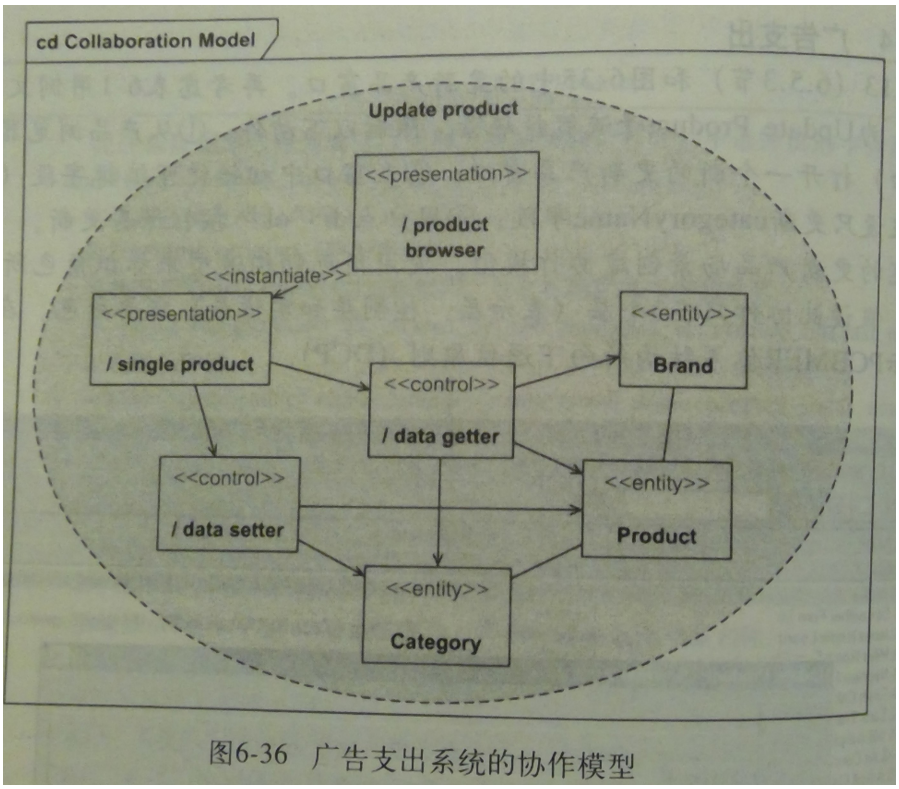



图6-36 广告支出系统的协作模型

e. 从交互到复合结构

i. 复合结构图是与协作范围所对应的一种类图。

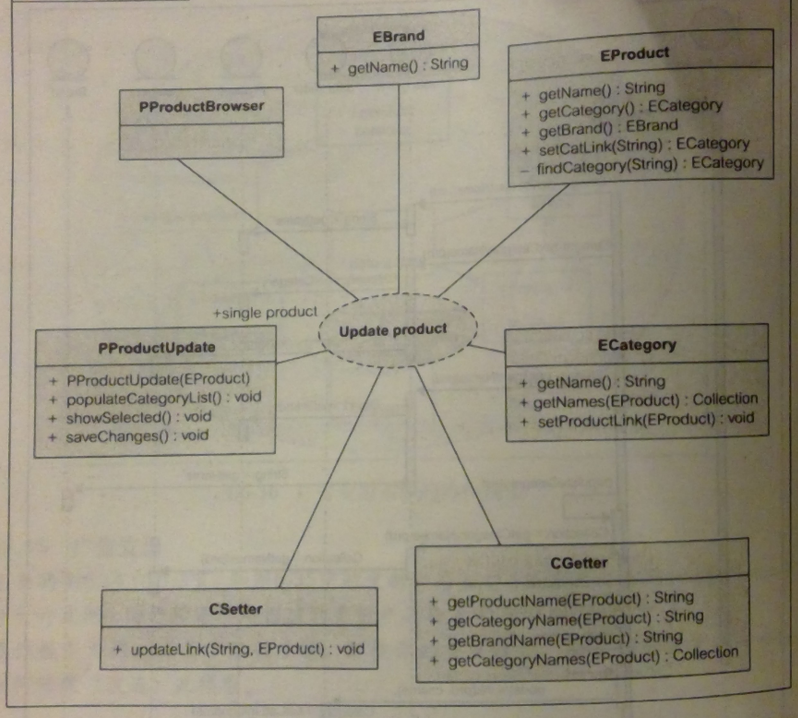



图6-38　广告支出系统的复合结构图